

Time Complexity

30 Practice Questions with Explained Answers

Section A — Iterative Loops (Q1 – Q13)

Q1. Determine the time complexity:

```
for(int i=0; i<n; i++)  
{  
    cout<<i;  
}
```

Answer:

The loop runs from $i = 0$ up to $i = n-1$, executing exactly n times. Each iteration does a constant amount of work.

Time Complexity: $O(n)$

Q2. Determine the time complexity:

```
for(int i=0; i<n; i++)  
{  
    for(int j=0; j<n; j++)  
    {  
        cout<<"*";  
    }  
}
```

Answer:

The outer loop runs n times. For each of those iterations, the inner loop also runs n times. Total operations = $n \times n$.

Time Complexity: $O(n^2)$

Q3. Determine the time complexity:

```
for(int i=0; i<n; i++)
{
    for(int j=0; j<i; j++)
    {
        cout<<"*";
    }
}
```

Answer:

When $i=0$ the inner loop runs 0 times, when $i=1$ it runs 1 time, and so on. Total = $0+1+2+\dots+(n-1) = n(n-1)/2$, which is quadratic.

Time Complexity: $O(n^2)$

Q4. Determine the time complexity:

```
for(int i=1; i<n; i*=2)
{
    cout<<i;
}
```

Answer:

i takes the values 1, 2, 4, 8, ... doubling each time. It reaches n after $\log_2(n)$ steps.

Time Complexity: $O(\log n)$

Q5. Determine the time complexity:

```
for(int i=n; i>1; i/=2)
{
    cout<<i;
}
```

Answer:

i starts at n and is halved each iteration: $n, n/2, n/4, \dots$. The loop ends when i reaches 1, after $\log_2(n)$ iterations.

Time Complexity: $O(\log n)$

Q6. Determine the time complexity:

```
for(int i=1; i<=n; i++)
{
for(int j=1; j<=log(n); j++)
{
cout<<"*";
}
}
```

Answer:

The outer loop runs n times. The inner loop runs $\log(n)$ times per outer iteration. Total operations = $n \times \log(n)$.

Time Complexity: $O(n \log n)$

Q7. Determine the time complexity:

```
while(n>0)
{
n--;
}
```

Answer:

n decrements by 1 each iteration, starting from its initial value and going down to 0. The loop executes exactly n times.

Time Complexity: $O(n)$

Q8. Determine the time complexity:

```
while(n>1)
{
n/=2;
}
```

Answer:

n is halved in each step: $n, n/2, n/4, \dots$. The process stops when n reaches 1, after $\log_2(n)$ divisions.

Time Complexity: $O(\log n)$

Q9. Determine the time complexity:

```
for(int i=1; i<=n; i++) { }  
for(int j=1; j<=n; j++) { }
```

Answer:

Both loops are sequential, not nested. The first contributes $O(n)$ and the second also $O(n)$. In total $O(n) + O(n) = O(n)$, since we drop the constant.

Time Complexity: $O(n)$

Q10. Determine the time complexity:

```
for(int i=1; i<=n; i++) { }  
for(int j=1; j<=n*n; j++) { }
```

Answer:

The loops run one after the other. First loop: $O(n)$. Second loop: $O(n^2)$. Total = $O(n) + O(n^2)$. The dominant term is n^2 .

Time Complexity: $O(n^2)$

Q11. Determine the time complexity:

```
for(int i=1; i<=n; i++)  
{  
  for(int j=1; j<=n*n; j++)  
  {  
  }  
}
```

Answer:

The outer loop runs n times. The inner loop runs n^2 times for each outer iteration. Total = $n \times n^2 = n^3$.

Time Complexity: $O(n^3)$

Q12. Determine the time complexity:

```
for(int i=1; i<=n; i*=3)
{
}
```

Answer:

i triples each time: 1, 3, 9, 27, ... The loop runs while $i \leq n$, so the number of iterations is $\log_3(n)$, which is still $O(\log n)$ since log base only changes by a constant factor.

Time Complexity: $O(\log n)$

Q13. Determine the time complexity:

```
for(int i=1; i<=n; i++)
{
  for(int j=1; j<=i*i; j++)
  {
  }
}
```

Answer:

The inner loop runs i^2 times for each i . Total = $1^2 + 2^2 + \dots + n^2 = n(n+1)(2n+1)/6$. This sum grows as n^3 .

Time Complexity: $O(n^3)$

Section B — Recursion (Q14 – Q20)

Q14. Determine the time complexity:

```
void fun(int n)
{
  if(n==0) return;
  fun(n-1);
}
```

Answer:

The function reduces n by 1 on each call, going from n down to 0. There are $n+1$ calls total, each doing $O(1)$ work. Recurrence: $T(n) = T(n-1) + 1$.

Time Complexity: $O(n)$

Q15. Determine the time complexity:

```
void fun(int n)
{
    if(n<=1) return;
    fun(n/2);
}
```

Answer:

The function halves n with every recursive call. The call stack depth is $\log_2(n)$. Recurrence: $T(n) = T(n/2) + 1$.

Time Complexity: $O(\log n)$

Q16. Determine the time complexity:

```
void fun(int n)
{
    if(n==0) return;
    fun(n-1);
    fun(n-1);
}
```

Answer:

Each call branches into two calls with $n-1$. The recursion tree has depth n and the number of nodes doubles at each level, giving $2^0 + 2^1 + \dots + 2^n$ total calls. Recurrence: $T(n) = 2T(n-1) + 1$.

Time Complexity: $O(2^n)$

Q17. Determine the time complexity:

```
int fib(int n)
{
    if(n<=1) return n;
    return fib(n-1) + fib(n-2);
}
```

Answer:

Each call spawns two more calls with slightly smaller inputs. The resulting recursion tree has approximately 2^n nodes. Recurrence: $T(n) = T(n-1) + T(n-2) + 1$.

Time Complexity: $O(2^n)$

Q18. Determine the time complexity:

```
int fact(int n)
{
    if(n<=1) return 1;
    return n * fact(n-1);
}
```

Answer:

A single recursive call is made with $n-1$ each time. The function is called n times before reaching the base case, and each call does $O(1)$ work. Recurrence: $T(n) = T(n-1) + 1$.

Time Complexity: $O(n)$

Q19. Determine the time complexity:

```
void fun(int n)
{
    if(n<=0) return;
    fun(n-1);
    cout<<n;
}
```

Answer:

The function recurses to the base case first, then prints on the way back up. The recursion depth is n and each level does $O(1)$ work. Recurrence: $T(n) = T(n-1) + 1$.

Time Complexity: $O(n)$

Q20. Determine the time complexity:

```
void fun(int n)
{
    if(n<=1) return;
    fun(n/2);
    cout<<n;
}
```

Answer:

The function halves n on each call before printing on the way back. The call stack depth is $\log_2(n)$ and each level contributes $O(1)$. Recurrence: $T(n) = T(n/2) + 1$.

Time Complexity: $O(\log n)$

Section C — Recurrence Relations (Q21 – Q25)

Q21. Solve the recurrence relation:

$$T(n) = T(n-1) + 1$$

Answer:

Expanding step by step: $T(n) = T(n-1)+1 = T(n-2)+2 = \dots = T(0)+n$. Assuming $T(0)$ is a constant, the solution grows linearly.

Time Complexity: $O(n)$

Q22. Solve the recurrence relation:

$$T(n) = T(n-1) + n$$

Answer:

Expanding: $T(n) = T(n-1)+n = T(n-2)+(n-1)+n = \dots = T(0)+1+2+\dots+n = n(n+1)/2 + \text{constant}$. This is quadratic.

Time Complexity: $O(n^2)$

Q23. Solve the recurrence relation:

$$T(n) = T(n/2) + 1$$

Answer:

Each expansion adds 1 and halves n . After $\log_2(n)$ expansions we reach the base case. By the Master Theorem (Case 2 with log factor 0), $T(n) = O(\log n)$.

Time Complexity: $O(\log n)$

Q24. Solve the recurrence relation:

$$T(n) = 2T(n-1) + 1$$

Answer:

Expanding: $T(n) = 2T(n-1)+1 = 4T(n-2)+3 = 8T(n-3)+7 = \dots = 2^n \cdot T(0) + (2^n - 1)$. The dominant term is 2^n .

Time Complexity: $O(2^n)$

Q25. Solve the recurrence relation:

$$T(n) = T(n/2) + n$$

Answer:

By the Master Theorem: $a=1$, $b=2$, $f(n)=n$. Since $f(n) = n$ grows faster than $n^{\log_2 1} = 1$ by a polynomial factor, Case 3 applies. $T(n) = O(n)$.

Time Complexity: $O(n)$

Bonus — GATE-Level Questions (Q26 – Q30)

Q26. Determine the time complexity:

```
for(int i=1; i<=n; i++)
{
    for(int j=1; j<=i; j*=2)
    {
    }
}
```

Answer:

The inner loop runs $\log_2(i)$ times for each value of i (since j doubles). Summing over all i : $\log(1)+\log(2)+\dots+\log(n) = \log(n!) \approx n \cdot \log(n)$ by Stirling's approximation.

Time Complexity: $O(n \log n)$

Q27. Determine the time complexity:

```
for(int i=1; i<=n; i*=2)
{
    for(int j=1; j<=n; j++)
    {
    }
}
```

Answer:

The outer loop runs $\log(n)$ times as i doubles each time. For each outer iteration, the inner loop runs n times. Total = $n \times \log(n)$.

Time Complexity: $O(n \log n)$

Q28. Determine the time complexity:

```
for(int i=n; i>=1; i--)  
{  
  for(int j=i; j>=1; j--)  
  {  
  }  
}
```

Answer:

The inner loop runs i iterations for each value of i . Total = $n+(n-1)+\dots+2+1 = n(n+1)/2$. This is the classic triangular sum, which is quadratic.

Time Complexity: $O(n^2)$

Q29. Determine the time complexity:

```
for(int i=1; i<=n; i++)  
{  
  for(int j=1; j<=n; j*=2)  
  {  
  }  
}
```

Answer:

The outer loop runs n times. The inner loop runs $\log(n)$ times per iteration since j doubles each time. Total = $n \times \log(n)$.

Time Complexity: $O(n \log n)$

Q30. Determine the time complexity:

```
void fun(int n)  
{  
  if(n<=1) return;  
  fun(n/2);  
  fun(n/2);  
}
```

Answer:

Two recursive calls are made, each with $n/2$. Recurrence: $T(n) = 2T(n/2) + 1$. By Master Theorem: $a=2$, $b=2$, $f(n)=1$. Since $\log_2(2)=1$, we have n^1 vs $f(n)=1$. Case 1 applies: $T(n) = O(n^{\log_2 2}) = O(n)$.

Time Complexity: $O(n)$
